

Introduction to LLMs

An Overview

Alexander Quispe

Social Sciences Faculty
Pontificia Universidad Católica del Perú

May 13, 2025

Credit

These slides are based on Grant Sanderson's *Neural Networks* series of videos.

Overview

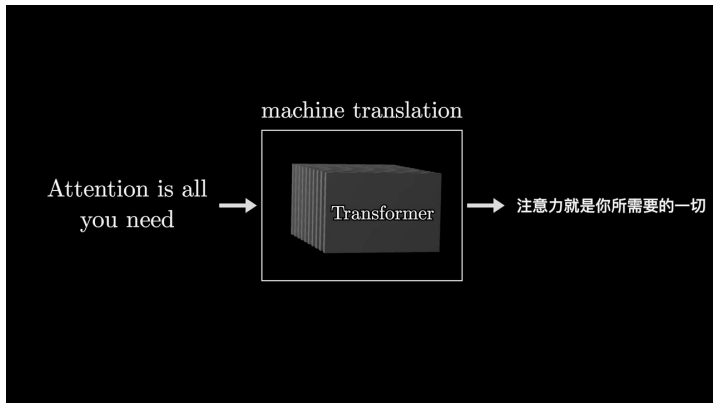
1. The Transformer Architecture

2. Tokenization

3. Embeddings

The Transformer Architecture

- *Transformers* were introduced in *Attention Is All You Need* by Vaswani et al. (2017) from Google
- Initially, they were focused on translation tasks.



Other Uses of Transformers

- Speech-to-text
- Text-to-speech
- Image generation¹
- Text completion

¹Usually when paired with diffusion, another type of architecture that is outside of the scope of this class

What Transformers Do

- Transformers output a **predicted probability distribution** of **tokens** that may follow a piece of text.
- To generate text, the LLM algorithm samples one token from that distribution

If you could see the underlying
probability distributions a large
language model uses when generating
text, then **you**



What Transformers Do

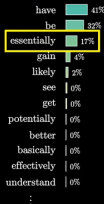
- This procedure is **recursive**; the output of the previous step is used as input for the next step

If you could see the underlying probability distributions a large language model uses when generating text, then you **would**



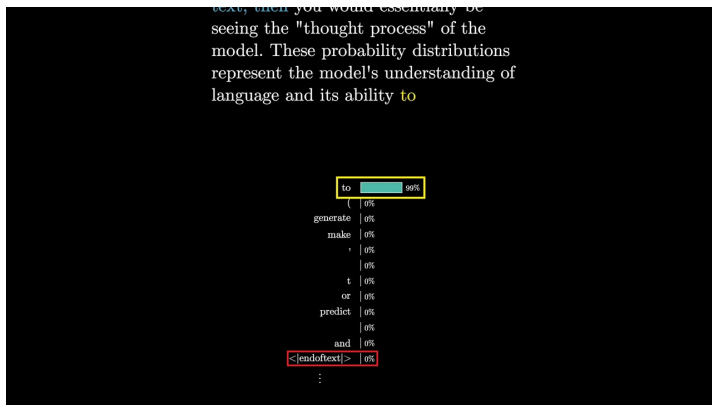
What Transformers Do

If you could see the underlying probability distributions a large language model uses when generating text, then you would **essentially**



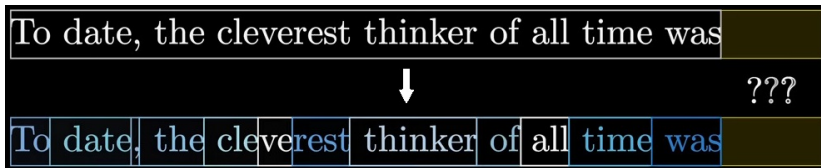
What Transformers Do

- It does this until it reaches a maximum number of tokens or an **end of text** token (enclosed in red in the following figure)



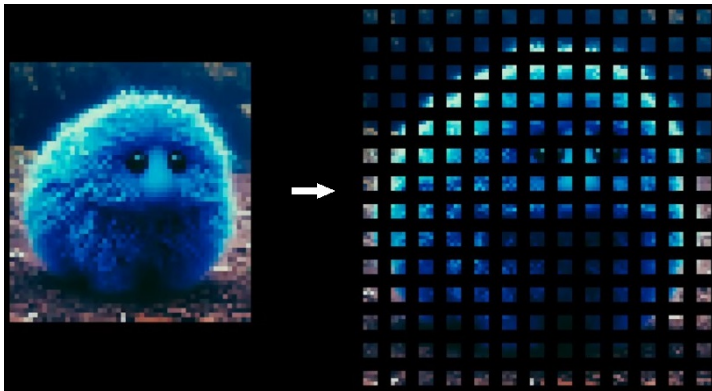
What Is a Token?

- We have mentioned that the transformer architecture outputs one sample from a predicted conditional probability distribution for **tokens**
- These tokens are small chunks of "data" that serve as inputs and outputs for the transformers
- They can be text tokens



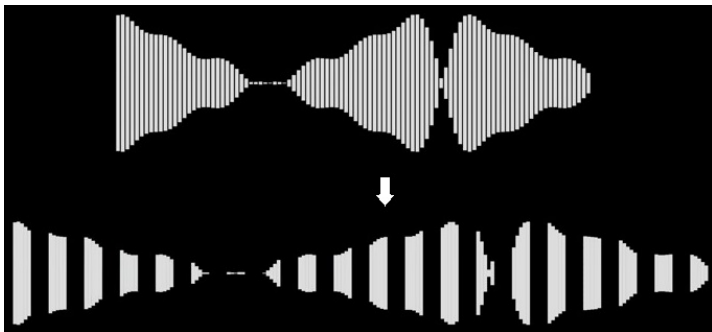
What Is a Token?

- Image tokens



What Is a Token?

- Or audio



A Convenient Lie

- From now on, we will refer to tokens as *words*
- It is easier to understand the following concepts this way
- However, **tokens are not words** in reality. They are closer to morphemes.

The Truth

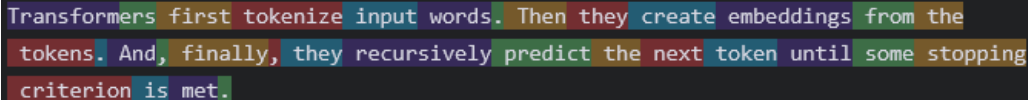
This process (known fancifully as tokenization) frequently subdivides words

A Convenient Lie

It's nice to sometimes pretend tokens are words

Tokens in the Transformer

- Within the transformer, each token is represented by a positive integer, the token's **ID**.
- You can play around with [OpenAI's example](#)
- Input text: *Transformers first tokenize input words. Then they create embeddings from the tokens. And, finally, they recursively predict the next token until some stopping criterion is met.*
- Resulting tokens:



Transformers first tokenize input words. Then they create embeddings from the tokens. And, finally, they recursively predict the next token until some stopping criterion is met.

- Token IDs: [12200, 409, 1577, 192720, 3422, 6391, 13, 7801, 1023, 2501, 174989, 591, 290, 20290, 13, 1958, 11, 8486, 11, 1023, 130266, 17946, 290, 2613, 6602, 4609, 1236, 36616, 71270, 382, 1421, 13]

Embeddings: Encoding Meaning

- After encoding, an LLM encodes each word's *meaning*
- How does it do this?

The Embedding Matrix

- One sub-component of modern LLMs is the embedding matrix, which we call W_E
- Each column vector in W_E corresponds to a word
- The (real-valued) entries in the vector serve to encode **semantic meaning**
- Each entry in W_E is a parameter that starts off randomly and is learned through data

All words, $\sim 50k$

aah	aardvark	aardwolf	aargh	ab	aback	abacterial	abacus	abalone	abandon	...	zygoid	zygomatic	zygomorphic	zygosis	zygote	zygotic	zyme	zymogen	zymosis	zzz
+1.0	+4.3	+2.0	+0.9	-1.5	+2.9	-1.2	+7.8	+9.2	-2.3	...	+0.6	+1.3	+8.4	-8.5	-8.2	-9.5	+6.6	+5.5	+7.3	+9.5
+5.9	-0.8	+5.6	-7.6	+2.8	-7.1	+8.8	+0.4	-1.7	-4.7	...	-0.9	+1.4	-9.5	+2.3	+2.2	+2.3	+8.8	+3.6	-2.8	-1.2
+3.9	-8.7	+3.3	+3.4	-5.7	-7.3	-3.7	-2.7	+1.4	-1.2	...	-7.9	-5.8	-6.7	+3.0	-4.9	-0.7	-5.1	-6.8	-7.7	+3.1
-7.2	-6.0	-2.6	+6.4	-8.0	+6.7	-8.0	+9.4	-0.6	+9.4	...	+4.7	-9.1	-4.3	-7.5	-4.0	-7.5	-3.6	-1.7	-8.6	+3.8
+1.3	-4.6	+0.5	-8.0	+1.5	+8.5	-3.6	+3.3	-7.3	+4.3	...	-6.3	+1.7	-9.5	+6.5	-9.8	+3.5	-4.6	+4.7	+9.2	-5.0
+1.5	+1.8	+1.4	-5.5	+9.0	-1.0	+6.9	+3.9	-4.0	+6.2	...	+7.5	+1.6	+7.6	+3.8	+4.5	+0.0	+9.0	+2.9	-1.5	+2.1
-9.5	-3.9	+3.2	-4.2	+2.3	-1.4	-7.2	-4.0	+1.4	+1.8	...	+3.0	+3.0	-1.4	+7.9	-2.6	-1.3	+7.8	+6.1	+4.0	-7.9
+8.3	+4.2	+9.9	-6.9	+7.3	-6.7	+2.3	-7.4	+6.9	+6.1	...	-1.8	-8.5	+3.9	-0.9	+4.4	+7.3	+9.4	+7.0	-9.7	-2.8
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
-3.7	-2.0	-5.7	-6.2	+8.8	+4.7	-0.2	-5.4	-4.9	-8.8	...	-3.7	+3.9	-2.4	-6.3	-9.4	-8.6	+3.6	-0.9	+0.7	+7.9

What "Embedding" Means

- Keeping in mind that each word corresponds to a vector, we can think of embeddings as representations of words in some (vector) space
- These spaces are very high-dimensional
- e.g. GPT-3's embedding matrix has 12 288 rows, and more recent models likely have orders of magnitude more
- We will be working with 3-dimensional projections in this presentation
- Also, we refer to the embedding of "word" as the function $E(\text{word})$

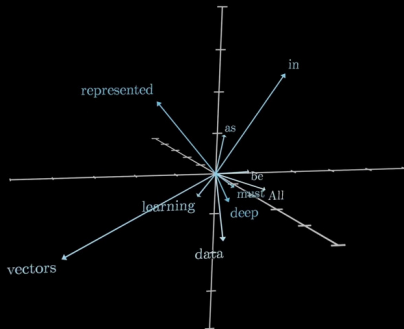
What "Embedding" Means

Words

→ "Embedding" →

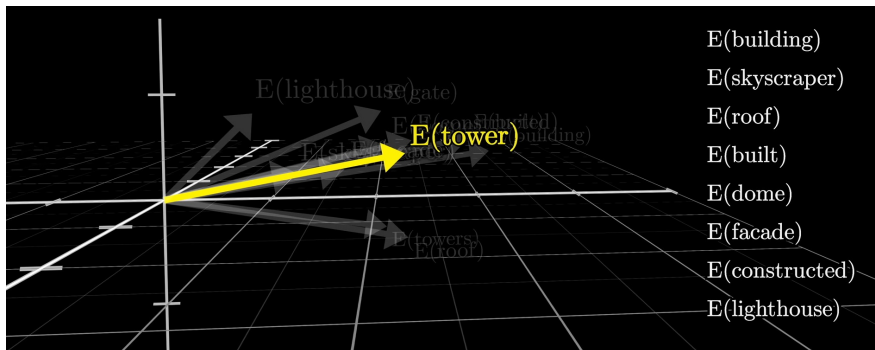
Vectors

All
data
in
deep
learning
must
be
represented
as
vectors



Semantic Meaning from Embeddings

- The embedding layer is learned such that each dimension (*direction*) can be interpreted as encoding semantic content (*meaning*) from words
- The following figure shows how words with related meanings have **similar** vectors (directions and magnitudes)

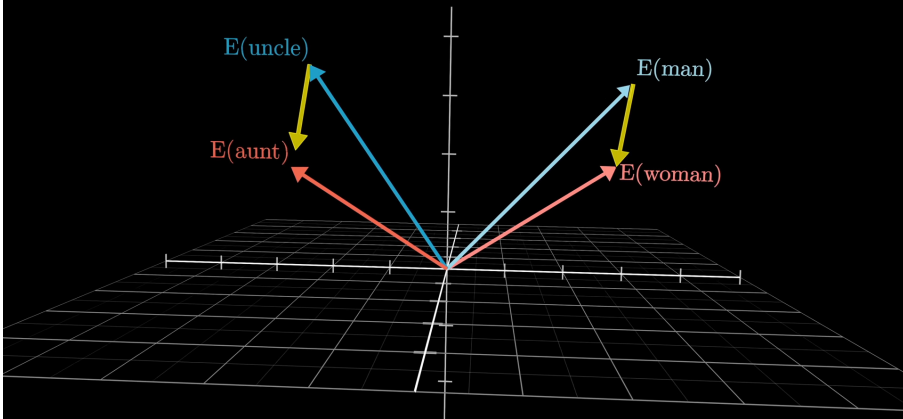


Vector Relationships

- Another feature of vector embeddings is that relationships between meanings are conserved.
- This means we can look at the relationship (difference) between two vectors A and B ($A - B$), apply it to another vector C (that is, compute $C + (A - B)$), and we will approximately get another vector D with a similar relationship to C as A has with B
- The following figure in the following slide makes this more clear

Vector Relationships

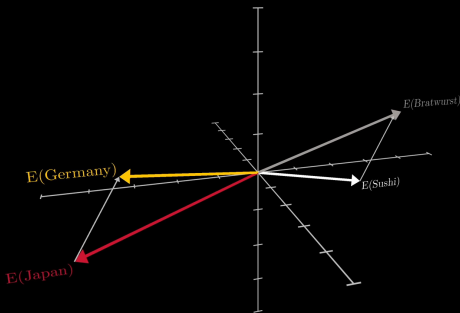
$$E(\text{aunt}) - E(\text{uncle}) \approx E(\text{woman}) - E(\text{man})$$



Another Interesting Example

- Embeddings can encode many dimensions of meaning
- In the following example, the vector space seems to have a dimension that encodes nationality, and another one that encodes notable foods.

$$E(\text{Sushi}) + E(\text{Germany}) - E(\text{Japan}) \approx E(\text{Bratwurst})$$



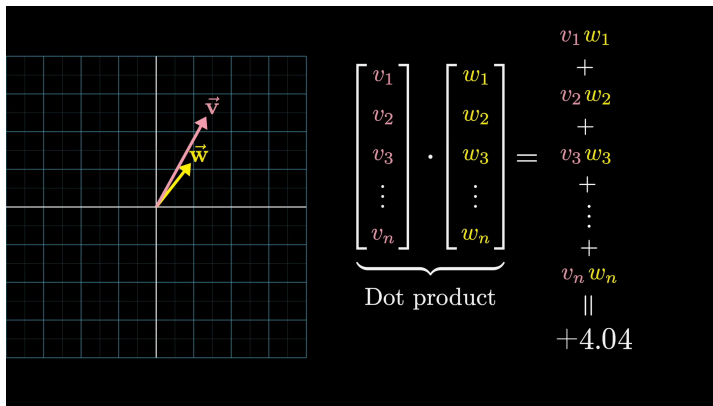
Small Digression: Dot Products

- The dot product between two vectors v and w (of the same size n) is defined as

$$v \cdot w = v_1 w_1 + v_2 w_2 + \dots + v_n w_n = \sum_{i=1}^n v_i w_i$$

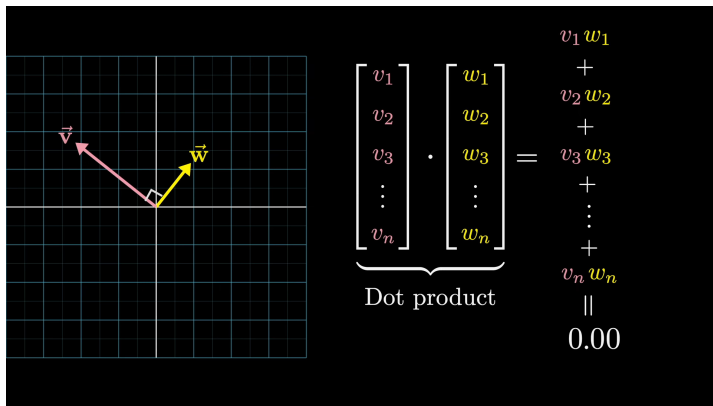
Small Digression: Dot Products

- Geometrically, when two vectors are close together, their dot product is positive



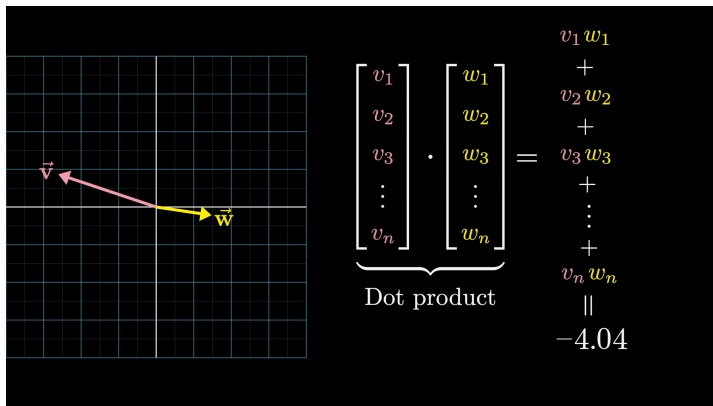
Small Digression: Dot Products

- When they're orthogonal, their dot product is zero



Small Digression: Dot Products

- And when they're far apart, their dot product is negative



The End